

Machine Learning Notes

Chapter 2: Differentiable Programming and Optimization

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 17, 2026

In Chapter 1, we established the forward pass—the mechanism by which a neural network transforms inputs into predictions. However, the network was static, relying on random initialization. In this chapter, we unlock the engine of learning. We explore how neural networks evaluate the impact of every parameter on the final error using differentiable programming (Autograd) and how they navigate the complex landscape of the loss function using optimization algorithms. We will build a miniature automatic differentiation engine from scratch to demystify the `.backward()` call found in modern frameworks.

Contents

<i>Part I: The Engine (Differentiable Programming)</i>	3
<i>The Calculus of Compositions: The Chain Rule</i>	3
<i>The Multivariable Chain Rule (The Sum Rule)</i>	3
<i>Computational Graphs</i>	4
<i>Backpropagation is Reverse-Mode Autodiff</i>	4
<i>A Complete Numeric Walkthrough</i>	4
<i>Forward-Mode vs. Reverse-Mode Autodiff</i>	5
<i>Forward-Mode Autodiff (Dual Numbers)</i>	5
<i>Reverse-Mode Autodiff (Backpropagation)</i>	6
<i>Implementing the "Micrograd"</i>	6
<i>Topological Sort and the Backward Pass</i>	9
<i>Building a Network with Valor</i>	10
<i>A Complete Training Example</i>	11
<i>Transition to PyTorch</i>	12
<i>From Scalars to Tensors: The Jacobian</i>	12
<i>Vector–Jacobian Products (VJPs)</i>	12
<i>Part II: The Steering Wheel (Optimization)</i>	13
<i>The Loss Landscape</i>	13
<i>The Training Loop</i>	14
<i>Gradient Descent vs. Stochastic Gradient Descent</i>	15
<i>The Learning Rate</i>	15
<i>The Evolution of Optimizers</i>	16
<i>Momentum</i>	16
<i>RMSProp (Adaptive Gradients)</i>	17
<i>Adam (Adaptive Moment Estimation)</i>	17
<i>Choosing an Optimizer</i>	18
<i>Exercises</i>	18

Part I: The Engine (Differentiable Programming)

To teach our model to learn, we must answer a fundamental question: *If I change this specific weight by a tiny amount, how much will the total error change?* The mathematical tool to answer this is the derivative. However, neural networks have millions of parameters nested in complex compositions. Calculating these derivatives manually is infeasible. This is where Automatic Differentiation (Autodiff) comes in.

The Calculus of Compositions: The Chain Rule

The foundation of autodiff is the chain rule from calculus. If a scalar variable L depends on y , and y depends on x , the rate of change of L with respect to x is the product of the local rates of change:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

A neural network is simply a massive composition of functions:

$L = f_n(f_{n-1}(\dots f_1(\mathbf{x}) \dots))$. The chain rule allows us to decompose the gradient of the loss with respect to the inputs into a product of local gradients calculated at each layer.

The Multivariable Chain Rule (The Sum Rule)

In deep learning, a parameter often influences the loss through *multiple paths* in the network. Suppose L depends on y_1 and y_2 , and both y_1 and y_2 depend on x . The total derivative of L with respect to x is the sum of the derivatives along each path:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial x}$$

This "sum rule" is critical. It means that during backpropagation, if a node receives gradients from multiple downstream paths, those gradients must be **accumulated** (summed) to find the true gradient.

Example 0.1 (A Variable That Appears Twice) Consider the expression $L = x^2 + x$ and suppose $x = 3$. Here x reaches the loss through two paths: the branch $y_1 = x^2$ and the branch $y_2 = x$. The local derivatives are

$$\frac{\partial y_1}{\partial x} = 2x = 6, \quad \frac{\partial y_2}{\partial x} = 1.$$

Since $L = y_1 + y_2$, both branches connect directly to the root with $\frac{\partial L}{\partial y_1} = \frac{\partial L}{\partial y_2} = 1$. The chain rule says we must add the contributions of every path:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial x} = 1 \cdot 6 + 1 \cdot 1 = 7.$$

We can verify analytically: $\frac{d}{dx}(x^2 + x) = 2x + 1 = 7$ at $x = 3$. Had we kept only one path, we would have obtained 6 or 1—both wrong. This is precisely why the *Valor* engine accumulates with $+=$: a parameter that is reused by many paths must receive the gradient from every one of them.

Computational Graphs

To automate the application of the chain rule, we represent the mathematical expressions as a **Computational Graph**—a Directed Acyclic Graph (DAG) where nodes are operations (or variables), and edges represent the flow of data.

Consider the expression $L = (a \times b) + c$. In a graph, inputs (a, b, c) are leaves, $d = a \times b$ is an intermediate node, $e = d + c$ is another, and $L = e$ is the root. The *forward pass* evaluates the graph from leaves to root, storing intermediate results.

Backpropagation is Reverse-Mode Autodiff

Backpropagation is simply the application of the chain rule to this graph in reverse—from the root (Loss) back to the leaves (Weights). We traverse the graph in **topological order**, ensuring that when we compute the gradient for a node, we have already accumulated all gradients from its children.

Let's trace it manually for Figure 1:

1. **At the root:** $\frac{\partial L}{\partial L} = 1.0$. We seed the graph with a gradient of 1.0.
2. **At node e :** Since $L = e$, locally $\frac{\partial L}{\partial e} = 1.0$. We pass this gradient backward.
3. **At node d and c (from e):** The operation is $e = d + c$. The local derivatives are $\frac{\partial e}{\partial d} = 1$ and $\frac{\partial e}{\partial c} = 1$. By the chain rule, $\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \times 1 = 1.0$, and $\frac{\partial L}{\partial c} = \frac{\partial L}{\partial e} \times 1 = 1.0$.
4. **At node a and b (from d):** The operation is $d = a \times b$. The local derivatives are $\frac{\partial d}{\partial a} = b$ and $\frac{\partial d}{\partial b} = a$. Applying the chain rule: $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial d} \times b = 1.0 \times 3.0 = 3.0$, and $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial d} \times a = 1.0 \times 2.0 = 2.0$.

A Complete Numeric Walkthrough

To make the mechanism concrete, let us execute the entire forward and backward pass for the graph in Figure 1, step by step, tracking the values as a computer would.

Forward pass (evaluate leaves $\rightarrow$ root, caching each intermediate value):

Backward pass (reverse order, accumulating $\frac{\partial L}{\partial \cdot}$ at each node):

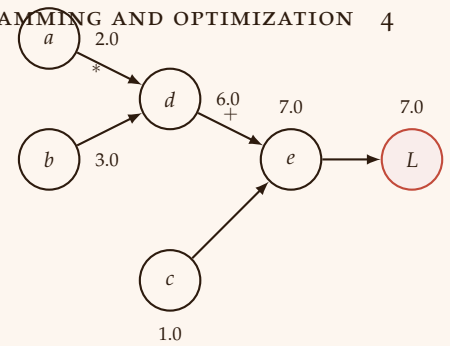


Figure 1: A computational graph for $L = (a \times b) + c$. Data flows forward (left to right). Gradients flow backward.

Step	Operation	Value	Stored in
1	$a = 2.0$	2.0	node a
2	$b = 3.0$	3.0	node b
3	$c = 1.0$	1.0	node c
4	$d = a \times b$	$2.0 \times 3.0 = 6.0$	node d
5	$e = d + c$	$6.0 + 1.0 = 7.0$	node e
6	$L = e$	7.0	node L

Table 1: Forward pass for $L = (a \times b) + c$ with $a = 2.0$, $b = 3.0$, $c = 1.0$.

Node	Local derivative	Incoming $\frac{\partial L}{\partial \cdot}$	Gradient
L	seed: $\frac{\partial L}{\partial L} = 1.0$	—	1.0
e	$\frac{\partial e}{\partial e} = 1.0$	$\frac{\partial L}{\partial e} = 1.0$	1.0
c	$\frac{\partial e}{\partial c} = 1.0$	$\frac{\partial L}{\partial c} = 1.0$	$1.0 \times 1.0 = 1.0$
d	$\frac{\partial e}{\partial d} = 1.0$	$\frac{\partial L}{\partial d} = 1.0$	$1.0 \times 1.0 = 1.0$
b	$\frac{\partial d}{\partial b} = a = 2.0$	$\frac{\partial L}{\partial b} = 1.0$	$1.0 \times 2.0 = 2.0$
a	$\frac{\partial d}{\partial a} = b = 3.0$	$\frac{\partial L}{\partial a} = 1.0$	$1.0 \times 3.0 = 3.0$

Table 2: Backward pass: each node multiplies its local derivative by the incoming gradient.

The final gradients are exactly what we derived by hand. The gradient $\left(\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b}, \frac{\partial L}{\partial c}\right) = (3.0, 2.0, 1.0)$ tells us the sensitivity of the loss to each input: perturbing a by a tiny amount ϵ changes L by about 3ϵ , perturbing b by 2ϵ , and perturbing c by only ϵ . In training, this is precisely the information used to decide how much each parameter should move.

The magic of Autodiff is that we don't write these equations manually. The framework builds the graph dynamically during the forward pass and traverses it in reverse to multiply and accumulate the local gradients automatically.

Forward-Mode vs. Reverse-Mode Autodiff

Backpropagation is one of two classical strategies for automatic differentiation. Understanding the alternative clarifies why neural networks use reverse mode.

Forward-Mode Autodiff (Dual Numbers)

Instead of storing a single number x , forward mode attaches to it a **dual number** $x + \dot{x}\epsilon$, where ϵ is a formal symbol with $\epsilon^2 = 0$. The component $\dot{x}$ tracks the derivative of x along a chosen direction. Arithmetic on dual numbers propagates derivatives for free:

$$(x + \dot{x}\epsilon) + (y + \dot{y}\epsilon) = (x + y) + (\dot{x} + \dot{y})\epsilon,$$

$$(x + \dot{x}\epsilon) \cdot (y + \dot{y}\epsilon) = (xy) + (\dot{x}y + x\dot{y})\epsilon.$$

Notice the ϵ -component of a product is exactly the product rule. If we seed $\dot{x} = 1$ for one input and $\dot{x} = 0$ for the others, a single forward

pass computes $\frac{\partial \mathcal{L}}{\partial x}$ for that one input. Computing the full gradient $\nabla_{\theta} \mathcal{L}$ over n parameters therefore requires n forward passes.

Reverse-Mode Autodiff (Backpropagation)

Backpropagation is the mirror image: it propagates derivatives from the output backward to the inputs. One forward pass evaluates all values; one backward pass computes the derivative with respect to every input simultaneously. Crucially, the cost of the backward pass is proportional to the number of edges in the graph—roughly the same as the forward pass—regardless of how many inputs there are.

Remark 0.1 (Why Reverse Mode Wins) *A neural network has millions of parameters θ (inputs to the derivative) but only one scalar loss $\mathcal{L}$ (the output). Forward mode would need n passes for n parameters; reverse mode needs exactly two. This single fact is why every deep learning framework computes gradients by backpropagation.*

	Forward Mode	Reverse Mode
What it computes	Derivative along one input direction	Derivative w.r.t. all inputs
Passes for n inputs	n forward passes	1 forward + 1 backward
Cost	$O(n \times \text{graph size})$	$O(\text{graph size})$
Best when	Few inputs, many outputs	Few outputs, many inputs
Deep learning	$n = \text{millions} \Rightarrow \text{hopeless}$	Exactly the training scenario

Table 3: Comparing forward-mode and reverse-mode automatic differentiation. Reverse mode is what PyTorch implements in `backward()`.

Implementing the “Micrograd”

To truly demystify PyTorch’s `.backward()`, we will build a minimal autograd engine. We create a `Valor` class that stores both the data and its gradient. When we perform operations on `Valor` objects, we dynamically build the computational graph.

Listing 1: The `Valor` class in `mini_nn/autograd.py`.

```

1 import numpy as np
2
3 class Valor:
4     def __init__(self, data, _children=(), _op=''):
5         self.data = data
6         self.grad = 0
7         # Internal variables for autograd graph
8         self._prev = set(_children)
9         self._op = _op

```

```

10     self._backward = lambda: None # Local gradient function
11
12     def __repr__(self):
13         return f"Valor(data={self.data:.4f}, grad={self.grad:.4f}
14             })"
15
16     def __add__(self, other):
17         # Ensure other is a Valor object
18         other = other if isinstance(other, Valor) else Valor(
19             other)
20         out = Valor(self.data + other.data, (self, other), '+')
21
22         def _backward():
23             # d(out)/d(self) = 1, d(out)/d(other) = 1
24             # We use += to accumulate gradients from multiple
25             # paths
26             self.grad += 1.0 * out.grad
27             other.grad += 1.0 * out.grad
28             out._backward = _backward
29             return out
30
31     def __mul__(self, other):
32         other = other if isinstance(other, Valor) else Valor(
33             other)
34         out = Valor(self.data * other.data, (self, other), '*')
35
36         def _backward():
37             # d(out)/d(self) = other, d(out)/d(other) = self
38             self.grad += other.data * out.grad
39             other.grad += self.data * out.grad
40             out._backward = _backward
41             return out
42
43     def __neg__(self):
44         # Negation is just multiplication by -1
45         return self * -1
46
47     def __sub__(self, other):
48         # Subtraction is addition of a negated term
49         return self + (-other)
50
51     def __rsub__(self, other):
52         # Handles expressions like 5 - x
53         return other + (-self)
54
55     def __truediv__(self, other):
56         # Division is multiplication by the inverse power
57         return self * other ** -1
58
59     def __pow__(self, other):
60         # only supports integer/float exponents

```

```

57     assert isinstance(other, (int, float)), "only constant
        powers"
58     out = Valor(self.data ** other, (self,), f'**{other}')
```



```

59
60     def _backward():
61         #  $d(x^n)/dx = n * x^{(n-1)}$ 
62         self.grad += (other * self.data ** (other - 1)) *
            out.grad
63     out._backward = _backward
64     return out
65
```



```

66     def exp(self):
67         x = self.data
68         out = Valor(np.exp(x), (self,), 'exp')
69
70     def _backward():
71         #  $d(\exp(x))/dx = \exp(x)$ 
72         self.grad += out.data * out.grad
73     out._backward = _backward
74     return out
75
```



```

76     def relu(self):
77         x = self.data
78         out = Valor(x if x > 0 else 0, (self,), 'relu')
79
80     def _backward():
81         #  $d(\text{relu}(x))/dx = 1 \text{ if } x > 0 \text{ else } 0$ 
82         self.grad += (out.data > 0) * out.grad
83     out._backward = _backward
84     return out
85
```



```

86     def sigmoid(self):
87         x = self.data
88         s = 1 / (1 + np.exp(-x))
89         out = Valor(s, (self,), 'sigmoid')
90
91     def _backward():
92         #  $d(\text{sigmoid}(x))/dx = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$ 
93         self.grad += s * (1 - s) * out.grad
94     out._backward = _backward
95     return out
96
```



```

97     def tanh(self):
98         x = self.data
99         t = (np.exp(2*x) - 1) / (np.exp(2*x) + 1)
100        out = Valor(t, (self,), 'tanh')
101
102    def _backward():
103        #  $d(\tanh)/dx = 1 - \tanh(x)^2$ 
104        self.grad += (1 - t**2) * out.grad
105    out._backward = _backward
```


106

`return out`

Every operation follows the same pattern: (1) build the output node from the inputs, (2) register the inputs as children, (3) define a `_backward` closure that implements the local derivative, and (4) attach it. Once an operation is defined, all the others compose through it—for example, subtraction is “add a negation” and division is “multiply by the inverse.” This composability is the whole trick: any differentiable function can be built from a small set of primitive operations.

Remark 0.2 (The += Accumulation) *Notice the use of += in the `_backward` functions. If a node is used multiple times in a graph (e.g., $y = x \times x$), the gradients from all paths must be summed. This is a direct application of the multivariable chain rule (the sum rule). Without +=, gradients would be overwritten instead of accumulated, leading to incorrect optimization.*

Topological Sort and the Backward Pass

To execute the backward pass, we must traverse the graph such that a node is processed only after all of its children have been processed. This ordering is called a **Topological Sort**.

Listing 2: The backward method using topological sort.

```

1  def backward(self):
2      # Build topological order (root must be last)
3      topo = []
4      visited = set()
5
6      def build_topo(v):
7          if v not in visited:
8              visited.add(v)
9              for child in v._prev:
10                 build_topo(child)
11                 topo.append(v)
12
13      build_topo(self)
14
15      # Seed the root gradient to 1.0
16      self.grad = 1.0
17
18      # Call backward() in reverse topological order
19      for v in reversed(topo):
20          v._backward()
```

This algorithm recursively visits children before parents, appending them to a list. By iterating over this list in reverse, we guarantee that when we calculate the local gradients of a node, the gradient flowing into it (`out.grad`) has already been fully accumulated.

Building a Network with Valor

We now have an engine that can differentiate any scalar expression. To build a neural network, we assemble Valor nodes into neurons and layers. Each neuron holds a vector of weights and a bias—all Valor objects—and computes the usual affine transformation followed by an activation:

Listing 3: A neuron, layer, and MLP built from Valor in `mini_nn/network.py`.

```

1  class Neuron:
2      def __init__(self, nin):
3          # nin = number of inputs; weights and bias are trainable
           Valor nodes
4          self.w = [Valor(np.random.uniform(-1, 1)) for _ in range
           (nin)]
5          self.b = Valor(0.0)
6
7      def __call__(self, x):
8          # z = sum(w_i * x_i) + b, then apply tanh
           z = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
9          return z.tanh()
10
11
12     def parameters(self):
13         return self.w + [self.b]
14
15 class Layer:
16     def __init__(self, nin, nout):
17         self.neurons = [Neuron(nin) for _ in range(nout)]
18
19     def __call__(self, x):
20         return [n(x) for n in self.neurons]
21
22     def parameters(self):
23         return [p for neuron in self.neurons for p in neuron.
           parameters()]
24
25 class MLP:
26     def __init__(self, sizes):
27         # sizes = [2, 4, 1] means 2 inputs, 4 hidden neurons, 1
           output
28         self.layers = [Layer(sizes[i], sizes[i+1]) for i in
           range(len(sizes) - 1)]
29
30     def __call__(self, x):
31         for layer in self.layers:
32             x = layer(x)           # outputs of one layer feed
           the next
33         return x[0]               # scalar output (single output
           neuron)

```

```

34
35     def parameters(self):
36         return [p for layer in self.layers for p in layer.
                    parameters()]

```

The beauty of this design is that the network is built the same way as any other expression. Calling an MLP on an input x performs the forward pass and silently constructs the computational graph. The `parameters()` method collects every `Value` node that should be optimized—these are exactly the leaves whose gradients backpropagation will compute.

A Complete Training Example

Let us put everything together by training a small MLP to learn the **XOR** function, the classic example of a problem that a single perceptron cannot solve. The full training loop exercises every concept from Part I and Part II:

Listing 4: Training a 2-4-1 MLP on XOR. Each step is a mini-batch update via SGD.

```

1  # Dataset: XOR (2 inputs, 1 output)
2  xs = [[0,0], [0,1], [1,0], [1,1]]
3  ys = [0, 1, 1, 0]
4
5  # Build the model and collect all parameters
6  model = MLP([2, 4, 1])
7  params = model.parameters()
8
9  learning_rate = 1.0
10 for epoch in range(200):
11     # ---- Forward pass ----
12     preds = [model(x) for x in xs]
13
14     # ---- Loss: mean squared error over the 4 examples ----
15     loss = sum((y - pred) ** 2 for y, pred in zip(ys, preds)) /
16           len(xs)
17
18     # ---- Backward pass ----
19     for p in params:
20         p.grad = 0.0          # zero_grad: clear accumulated
21                               gradients
22     loss.backward()
23
24     # ---- Update: plain SGD ----
25     for p in params:
26         p.data -= learning_rate * p.grad
27
28     if epoch % 40 == 0:
29         print(f"epoch {epoch:3d} | loss {loss.data:.4f}")

```

Remark 0.3 (Why `zero_grad()`?) Gradients accumulate (the sum rule!). If we did not reset `p.grad` to zero before each `backward()`, the gradients from previous mini-batches would keep piling up, and every update would overshoot. In PyTorch this is exactly what `optimizer.zero_grad()` does before `loss.backward()`.

Over 200 epochs the loss decreases toward zero and the network learns the XOR mapping—even though the two inputs are not linearly separable. This tiny example contains the entire lifecycle of modern deep learning: forward pass, loss, backward pass, and a parameter update driven by the gradient.

Transition to PyTorch

Our `Valor` class is functionally identical to PyTorch’s `torch.Tensor` in its autograd mechanics. When you call `loss.backward()` in PyTorch, it performs this exact reverse-mode autodiff on a highly optimized C++ graph.

From Scalars to Tensors: The Jacobian

However, PyTorch operates on *tensors* (multidimensional arrays) rather than scalars. The chain rule generalizes cleanly. If a function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ maps a vector input to a vector output, its derivative is no longer a single number but a matrix of all partial derivatives—the **Jacobian**:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{m \times n}$$

For a composition $\mathbf{h} = \mathbf{g} \circ \mathbf{f}$, the chain rule becomes a matrix product of Jacobians:

$$\mathbf{J}_{\mathbf{h}}(\mathbf{x}) = \mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x})) \cdot \mathbf{J}_{\mathbf{f}}(\mathbf{x})$$

Exactly the same graph structure as before—each edge now carries a matrix instead of a scalar.

Vector–Jacobian Products (VJPs)

Naively, the backward pass would multiply full Jacobian matrices, which are enormous. Fortunately, we do not need the entire matrix. In reverse mode, the gradient flowing back is a vector $\mathbf{v}$, and what we need from each node is the **Vector–Jacobian Product**

$$\mathbf{v}^T \mathbf{J}_{\mathbf{f}}(\mathbf{x}).$$

For any operation, autograd only needs to know how to multiply an incoming gradient vector by the operation’s Jacobian—never to materialize the Jacobian itself. For elementwise operations (like $\tanh$ applied to every entry of a tensor), the Jacobian is diagonal and the VJP is just an elementwise product, which is why $\tanh$ ’s `_backward` used $1 - \tanh^2$. This is the secret behind PyTorch’s efficiency: scalar autodiff (our Valor) and tensor autodiff (PyTorch) are the same algorithm, differing only in that the latter batches the arithmetic into matrices.

Remark 0.4 (Gradients of Losses Are Always Vectors) *Because the loss $\mathcal{L}$ is a scalar, $\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$ for any tensor $\mathbf{x}$ is a vector of the same shape as $\mathbf{x}$. This is why `param.grad` always “looks like” `param`. The scalar chain rule is simply applied entrywise through every tensor in the network.*

Part II: The Steering Wheel (Optimization)

Now that we possess the gradients—the vector pointing in the direction of steepest ascent of the loss—we must decide how to use this information to update the weights. This is the domain of optimization. If Autograd is the engine that calculates the slope, Optimization is the steering wheel that decides how fast and in what manner we descend the mountain.

The Loss Landscape

Imagine the loss function $\mathcal{L}(\theta)$ as a physical landscape in a space with millions of dimensions (one for each parameter in θ). The goal of training is to find the coordinates θ^* that minimize this landscape (the lowest valley). The gradient $\nabla_{\theta} \mathcal{L}$ tells us the direction of the steepest uphill slope. Therefore, to descend, we step in the exact opposite direction: $-\nabla_{\theta} \mathcal{L}$.

Unlike a smooth bowl, real loss landscapes contain rich structure that shapes how optimizers behave:

- **Global/Local Minima:** The valleys we aim for. A local minimum is the lowest point of its neighborhood but not necessarily of the whole landscape. Optimizers can get trapped in a local minimum and never find the global one.
- **Saddle Points:** Points where the slope is zero but which are neither a peak nor a valley—flat in one direction, sloping in another. Gradients vanish there, and plain SGD can stall for a long time.
- **Plateaus:** Large flat regions where gradients are near zero. Progress becomes extremely slow, which is why adaptive optimizers (which scale the step size) were invented.

Intuition: The Blind Hiker

Imagine a blindfolded hiker trying to reach the bottom of a valley. They feel the slope under their feet (the gradient) and take a step downhill (the update). The size of their step is the learning rate. If the step is too large, they might overshoot the valley and climb the opposite hill. If it’s too small, they will never reach the bottom before winter.

- **Ravines:** Narrow, steep valleys where the surface curves sharply in one direction and gently in another. They cause plain SGD to oscillate violently across the walls.

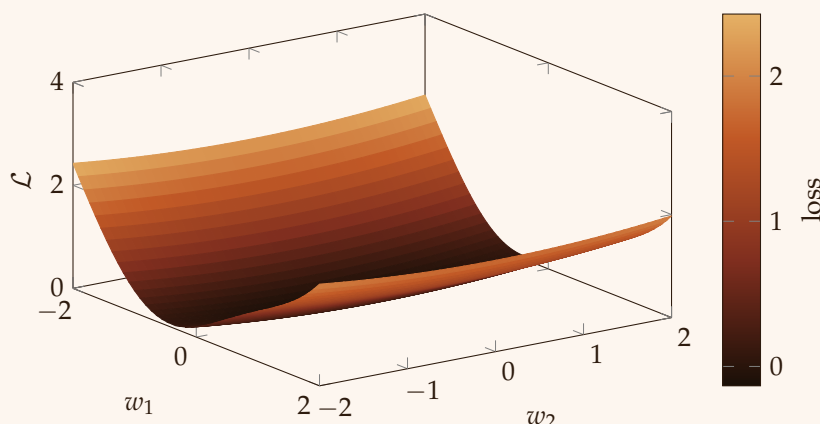


Figure 2: A slice of a loss landscape. The overall curvature of w_1 dominates, but local wrinkles create additional minima, and the shallow w_2 direction produces a ravine-like structure.

The blind hiker metaphor now becomes sharper: the gradient points steepest-uphill locally, and the update rule is the hiker's strategy for choosing step sizes in response to the terrain around them.

The Training Loop

The lifecycle of learning is a repetitive cycle:

1. **Forward Pass:** Feed a batch of data through the network to get predictions $\hat{y}$.
2. **Loss Calculation:** Compare $\hat{y}$ to targets y using $\mathcal{L}$.
3. **Backward Pass:** Run `backward()` to calculate $\frac{\partial \mathcal{L}}{\partial \theta}$ for all parameters θ .
4. **Update:** Adjust the weights using an optimizer.
5. **Zero Gradients:** Clear the gradient buffers to prepare for the next batch.

In PyTorch, this translates to the ubiquitous training step:

Listing 5: The standard PyTorch training step.

```

1 optimizer.zero_grad() # Step 5: Clear old gradients
2 loss = criterion(y_hat, y) # Steps 1 & 2: Forward and Loss
3 loss.backward() # Step 3: Autodiff
4 optimizer.step() # Step 4: Update weights

```

Why `zero_grad()`? In PyTorch, gradients are accumulated by default (due to the sum rule). If we don't clear them, the gradients

from step t will be added to the gradients of step $t - 1$, causing the network to take artificially large and chaotic steps.

Gradient Descent vs. Stochastic Gradient Descent

Batch Gradient Descent calculates the loss and gradients using the *entire* dataset at once. While mathematically precise (it computes the true gradient of the empirical risk), this is computationally prohibitive for datasets with millions of samples.

Stochastic Gradient Descent (SGD) uses a single random example per update. This is extremely fast but highly noisy. The loss will fluctuate violently, and the path to the minimum will resemble a random walk rather than a straight line.

The industry standard is **Mini-Batch SGD**. We process a small subset of data (e.g., 32, 64, or 256 samples) per update. This balances the stability of true gradients with the hardware efficiency of parallel matrix multiplications on GPUs.

	Batch GD	Mini-Batch SGD	Stochastic SGD
Samples per step	entire dataset	small subset (32–256)	a single sample
Gradient quality	exact (true gradient)	low-variance estimate	very noisy estimate
Updates per epoch	1	$\approx$ samples/-batch	samples
Speed	slow (one step/epoch)	fast	fastest
Stability	very stable, can get stuck	good balance	oscillates a lot
GPU efficiency	memory-heavy	excellent (batched mat-muls)	poor

Table 4: Comparison of gradient descent variants. Mini-batch SGD combines the precision of batch gradients with the throughput of hardware parallelism, making it the default in practice.

A useful intuition: the mini-batch gradient is a noisy estimate of the true gradient, and the batch size controls the noise level. Smaller batches mean more noise—which can actually help escape shallow local minima and saddle points—but also a more erratic training curve.

The Learning Rate

The learning rate η is the single most sensitive hyperparameter in deep learning. It controls the size of each step, and its interaction with the loss landscape determines whether training converges at all.

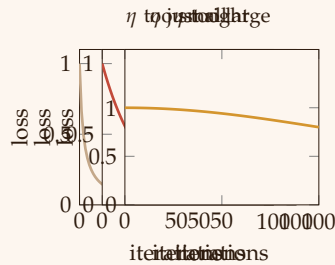


Figure 3: Effect of the learning rate on training. Too small $\Rightarrow$ painfully slow convergence; just right $\Rightarrow$ rapid smooth decrease; too large $\Rightarrow$ the loss oscillates or diverges.

A learning rate that is too large can cause divergence: each step overshoots the valley and lands higher up the opposite slope. A rate that is too small wastes compute. This is why modern practice uses learning rate schedules—start with a larger η and shrink it over time—or adaptive optimizers that effectively tune a per-parameter learning rate (Section).

The Evolution of Optimizers

Plain SGD updates weights using the rule: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$, where η is the learning rate. However, the loss landscape of deep neural networks is rarely a perfect bowl; it is full of ravines, saddle points, and flat plateaus. Plain SGD oscillates heavily in ravines (narrow steep valleys) and struggles to escape flat plateaus.

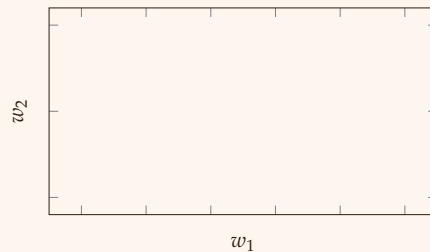


Figure 4: Optimization in a ravine. Plain SGD (terracotta) oscillates violently across the steep walls. Momentum (gold, dashed) averages out the oscillations, directing the path straight to the minimum.

Momentum

Momentum addresses the ravine problem by adding physics to the update. Instead of relying solely on the current gradient, it accumulates a velocity vector v_t :

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}, \quad \theta \leftarrow \theta - v_t$$

Here, γ (typically 0.9) acts as friction. If the gradient constantly changes direction (as in the zig-zag in Fig. 4), the perpendicular components of v_t cancel out, while the component pointing down the ravine accumulates, accelerating convergence.

Think of a ball rolling down a hill: it does not stop at every bump; it carries velocity from previous slopes. The friction coefficient γ decides how much past momentum to remember. A small γ behaves almost like plain SGD; a γ close to 1 (e.g., 0.99) remembers a long history and can overshoot past the minimum, requiring careful tuning. In practice, $\gamma = 0.9$ is a robust default.

RMSProp (Adaptive Gradients)

While momentum speeds up the descent, it still uses a global learning rate for all parameters. RMSProp adapts the learning rate for each parameter individually by dividing by the moving average of squared gradients:

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla_{\theta} \mathcal{L})^2, \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla_{\theta} \mathcal{L}$$

If a parameter has consistently large gradients, v_t becomes large, shrinking its effective learning rate. This prevents single parameters from causing destructive updates.

The intuition: some parameters may need large steps while others need tiny ones (e.g., rarely active parameters accumulate small gradients). Dividing each parameter's step by the root mean square of its recent gradients normalizes the step sizes across the whole network. The small constant ϵ (typically 10^{-8}) prevents division by zero. The decay factor β (typically 0.9) controls the window of recent gradients used for normalization.

Adam (Adaptive Moment Estimation)

Adam is the default optimizer in modern deep learning. It combines the concepts of Momentum (first moment) and RMSProp (second moment). It maintains an exponential moving average of both the gradients and the squared gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L} \quad (\text{First Moment})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2 \quad (\text{Second Moment})$$

Because m_t and v_t are initialized to zero, they are heavily biased toward zero in the early steps. Adam corrects this bias:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The final update rule is:

$$\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Adam adapts the learning rate for *each parameter individually*. Parameters that have had large gradients recently will have their learning rate reduced (via division by $\sqrt{\hat{v}_t}$), while parameters with small gradients will take larger steps. This makes Adam incredibly robust across different architectures and hyperparameters.

The bias correction is easy to misunderstand, so it is worth a concrete look. Suppose $\beta_1 = 0.9$ and the very first gradient is $g_1 = 1.0$. Then $m_1 = (1 - 0.9) \cdot 1.0 = 0.1$ —one tenth of the true gradient! The correction divides by $1 - \beta_1^1 = 0.1$, recovering $\hat{m}_1 = 1.0$. Without it, the first few updates would be far too small, artificially slowing the start of training. The default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across almost all problems.

Choosing an Optimizer

Optimizer	Key Idea	Strength	Weakness
SGD	follow $-\nabla \mathcal{L}$	simple, predictable	slow in ravines; sensitive to η
Momentum	accumulate velocity	escapes ravines, smooths path	adds γ ; can overshoot
RMSProp	per-parameter scaling	handles sparse/skewed grads	relies on good β
Adam	moments + bias correction	robust default; per-parameter rate	more hyperparameters

Table 5: A practical guide to the optimizers covered in this chapter. When in doubt, start with Adam; for fine-tuned convergence or memory constraints, Momentum SGD with a schedule is a strong choice.

Remark 0.5 (The End of the Engine) *We have now completed the core learning mechanism. Our `mini_nn` framework can build graphs, compute forward passes, calculate losses, backpropagate gradients, and update weights via Adam. However, we have ignored a critical problem: models with millions of parameters will simply memorize the training data. In Chapter 3, we will address Generalization and Regularization.*

Exercises

1. Derive the `_backward` function for a new primitive operation, the sine: $\frac{d}{dx} \sin(x) = \cos(x)$. Show how you would implement it following the same four-step pattern used in the `Valor` class.
2. Trace the backward pass for the expression $L = (x \times y) + x$ with $x = 2$ and $y = 3$ by hand. Note that x appears twice; verify that your result matches the analytic derivative $\frac{\partial L}{\partial x} = y + 1$.

3. The `Valor.backward()` method seeds `self.grad = 1.0` before the loop. Why must the root gradient be seeded, and what would happen if it were initialized to 0 instead?
4. Explain, in your own words, why reverse-mode autodiff is used for training while forward-mode would be hopeless. How many forward passes would forward-mode need for a model with 10^6 parameters and one scalar loss?
5. Using the sum rule, explain why `optimizer.zero_grad()` is necessary in the training loop. What incorrect gradient would a network compute if gradients were never reset?
6. Compare Batch GD, Mini-Batch SGD, and Stochastic SGD in terms of the noise in their gradient estimates. Which one can help escape a saddle point, and why?
7. Adam's first moment is initialized to zero and corrected via $1 - \beta_1^t$. Compute $\hat{m}_1$ and $\hat{m}_2$ for $\beta_1 = 0.9$ assuming a constant gradient $g = 1.0$, and confirm that the correction makes the early steps approximately unbiased.